BATMAN-ADV MESH NETWORKING, DRONECAN-BASED ESC FIRMWARE & SAD-BASED OPTICAL FLOW STABILIZATION ON LOW-COST EMBEDDED HARDWARE

Arya Sanchihar (Roll No. S260036) • Dept. of Electronics & Communication Engineering, RTU Rajasthan
SURGE Intern, Department of Electrical Engineering, IIT Kanpur • Mentor: Dr. Ketan Rajawat

COMMS ACTUATION VISION

MOTIVATION

Centralized infrastructure cellular networks, GPS, cloud compute is a liability in contested, remote, or cost-constrained environments: it's a single point of failure and a resource most cheaper embedded platforms can't assume. This work presents three independent, hardware-validated responses: a self-healing mesh network for infrastructure-free video, sensorless motor control requiring no encoder hardware, and a real-time vision pipeline running on a cheaper, resource-limited MCU. Each was built to answer the same question. How much resilience and capability can constrained, low-cost hardware deliver on its own?

BATMAN-ADV MESH VIDEO NETWORK

Infrastructure-independent P2P streaming across Raspberry Pi nodes

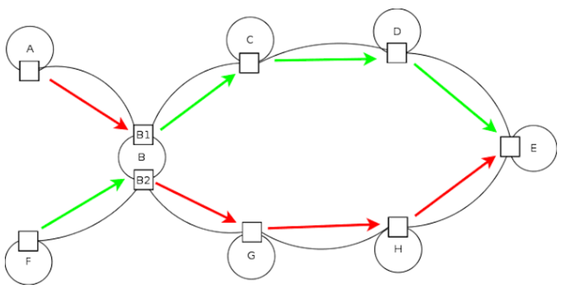
PROBLEM STATEMENT

Centralized comms (cellular, fixed Wi-Fi APs) create a single point of failure in a contested environment. A soldier-worn camera and multi-Pi mesh must route peer-to-peer video and telemetry with no central control, gracefully handling nodes joining, leaving, or moving out of range.

METHODOLOGY

- Layer-2 mesh: B.A.T.M.A.N.-Adv over ad-hoc wlan0, joined into a bat0 virtual-switch interface
- Zero-config addressing + routing: Avahi autoipd for link-local IPs (no DHCP), batman-adv for layer-2 next-hop forwarding
- Video path: GStreamer producer/consumer, H.264 over RTP/UDP; GTK based monitoring UI
- Validated first in a 3-container Docker simulation, then on 3 physical Raspberry Pi nodes (Pi 4 + Pi 5) with on-node LCD displays

RESULT



Illustrative routing topology (6-node example) showing best-next-hop selection logic; validated on 3 physical Raspberry Pi nodes.



A custom GTK4 desktop client drives the GStreamer consumer pipeline (rtph264depay → avdec_h264 → gtksink) for live per-node video preview. Each Pi runs the matching producer pipeline (v4l2src → x264enc → rtph264pay → udpsink) that feeds the mesh.

NOVELTY

Operates at the data-link layer, not IP, the whole mesh presents as one virtual switch, so every node need only find its best next hop rather than compute a full path. Combined with DHCP-free Avahi addressing, this gives fully self-organizing, infrastructure-independent bring-up, with each node doubling as both a relay and a local display endpoint.

DRONECAN-STAGED ESC FIRMWARE

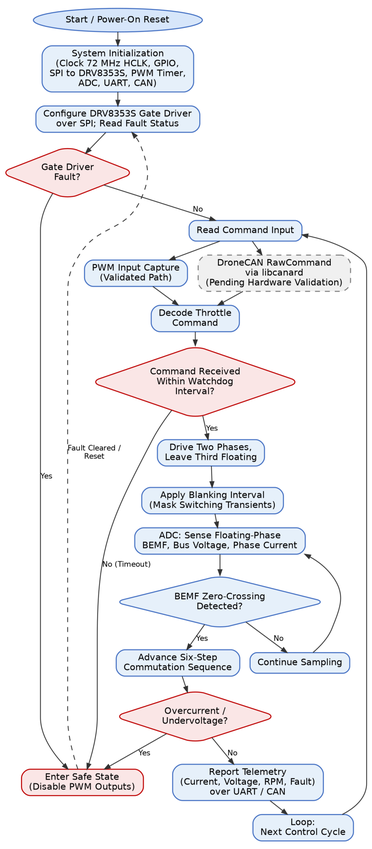
Sensorless BLDC control on STM32F303RET6 + DRV8353S

PROBLEM STATEMENT

Sensorless BLDC control eliminates Hall/encoder wiring, but most hobby ESCs speak only PWM — no standardized telemetry, no flight-controller integration. This firmware needed sensorless commutation, a CAN-based command and telemetry bus (DroneCAN) for compatibility with any DroneCAN flight controller, and safety-critical fault handling with watchdog-driven command timeout protection.

METHODOLOGY

- Hardware: STM32F303RET6 microcontroller (72 MHz) paired with a DRV8353S 3-phase gate driver, communicating over SPI.
- Commutation: sensorless six-step trapezoidal control using back-EMF zero-crossing detection, switching above 34 kHz (ARR = 2099).
- Command path: currently driven by PWM input, with DroneCAN RawCommand support already validated in software via libcanard, ready for hardware integration.
- Throttle ramping limited to 3 counts/ms for smooth acceleration, backed by a watchdog timer that forces a safe shutdown state if commands stop arriving.



RESULT

- Throttle @ g_throttle = 4095 (full-scale, 12-bit) → 100% commanded output
- PWM out @ g_pwm_pulse_us = 1500 µs, g_pwm_signal_valid = 1 → confirmed active and valid
- Timer capture @ TIM4→CNT = 2137, TIM4→CCR1 = 1500 → matches expected 1500-count pulse width
- Switching @ >34 kHz, 6-channel complementary gate drive

DroneCAN command handling and six-step sensorless commutation are validated on real hardware; safety fault handling is implemented but fault-injection testing is still pending.

NOVELTY

The firmware integrates sensorless commutation, DroneCAN command handling, and safety logic into one cohesive control loop rather than treating CAN as a bolt-on afterthought. A watchdog-driven command-timeout check runs independently of DRV8353S hardware fault detection, so the ESC can distinguish between "the bus went quiet" and "the driver itself failed" and respond to each correctly. This separation of concerns means future upgrades, additional telemetry fields, new DroneCAN message types, or a flight-controller-specific feature that can be layered on top without touching the validated commutation or safety paths.

SAD OPTICAL FLOW + EIS

Real-time adjacent stabilization on a 64 KB RAM Cortex-M4

PROBLEM STATEMENT

Dense optical flow (Lucas-Kanade, Horn-Schunck) needs iterative solving and heavy CPU/RAM - infeasible on a Cortex-M4 with 64 KB RAM and no hardware float throughput. A stabilized real-time feed still needs reliable per-frame motion estimation on this hardware.

METHODOLOGY

- SAD block-matching, integer-only, accelerated with Cortex-M4 USAD8 (4 abs-diffs/cycle)
- Pipeline: variance-based low-texture rejection → full-search SAD → histogram outlier filter → subpixel parabolic refinement → 2D Kalman smoothing → EIS crop-correction
- Benchmarked on STM32F303RE (Nucleo-64) against the MPI-Sintel dataset (Middlebury "Army" had insufficient motion for reliable detection)
- Compiler opt: -O2, reduced search radius (ρ = 6) and variance threshold (50 → 15) tuned for the 64 KB RAM budget

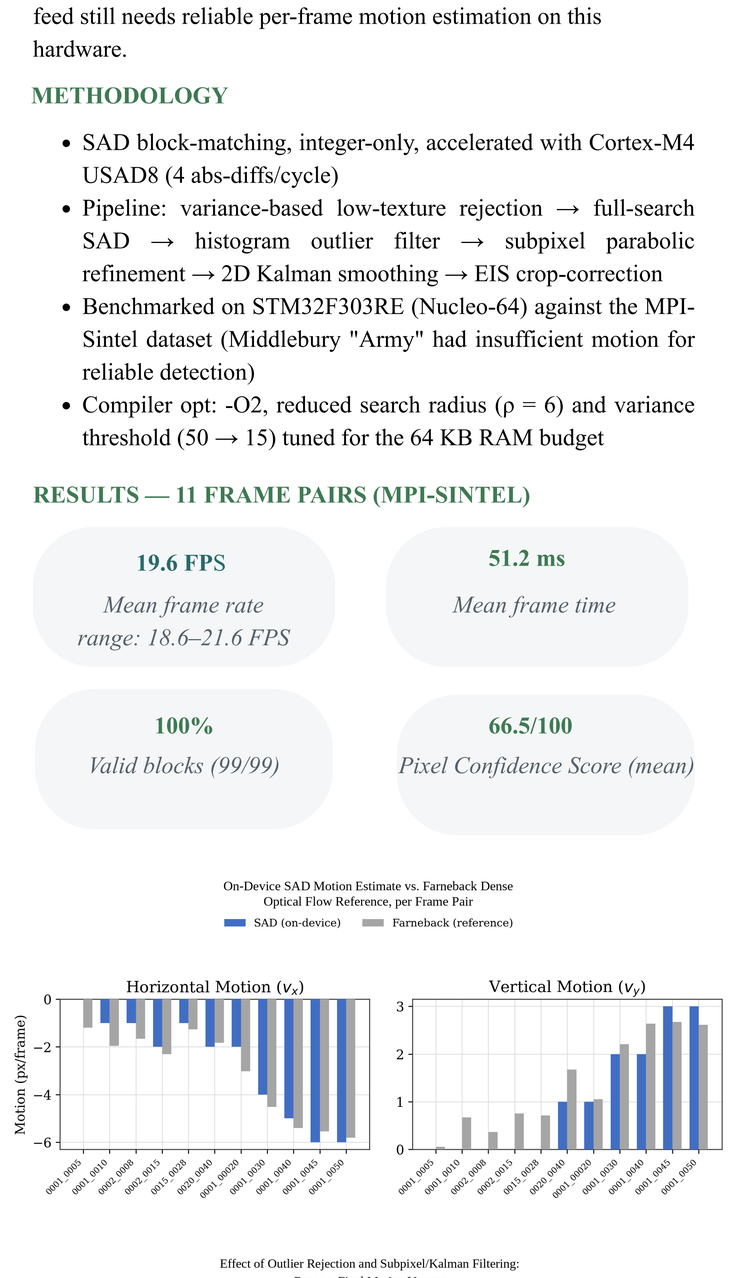
RESULTS — 11 FRAME PAIRS (MPI-SINTEL)

19.6 FPS
Mean frame rate range: 18.6–21.6 FPS

51.2 ms
Mean frame time

100%
Valid blocks (99/99)

66.5/100
Pixel Confidence Score (mean)



NOVELTY

Runs a complete 6-stage optical-flow + EIS pipeline not just raw block matching at a sustained ~19.6 FPS, with Cortex-M4 USAD8-accelerated integer SAD replacing a dense float solver. A 7-10 px EIS crop-correction margin absorbs the estimated motion without exposing frame edges, while search-radius and variance tuning trade a small, quantified accuracy margin for throughput headroom under a hard 64 KB RAM ceiling.

System Integration & Conclusion

These three projects were built and tested independently, but share one engineering principle: decouple a subsystem's core logic from infrastructure that may not yet be available. The mesh network was validated via Docker simulation before physical hardware; the ESC firmware was validated over PWM before its DroneCAN path could be tested. Only the mesh network is designed for the soldier-wearable use case , ESC firmware and optical flow pipeline are standalone embedded systems projects with broader applicability.

FUTURE WORK

- Mesh network: Multi-node field testing under realistic RF interference; extend range to 5–10 km via directional/sub-GHz radios with multi-hop batman-adv relay
- ESC firmware: End-to-end DroneCAN RawCommand validation on CAN-capable hardware; migrate control from six-step to field-oriented control (FOC)
- Optical flow: Fuse IMU (gyro/accelerometer) data with the SAD motion estimate to disambiguate platform rotation from translation and improve stabilization under fast motion; validate on outdoor drone-flight datasets with lighting variation, and higher-magnitude motion than MPI-Sintel

REFERENCE

- Neumann et al., "B.A.T.M.A.N.," IETF Draft, 2008.
- PX4/Zubax, "Sapog ESC Firmware," docs.px4.io/main/en/dronecan/sapog
- Butler et al., "MPI-Sintel," ECCV, 2012
- Open Mesh Project, "B.A.T.M.A.N. Advanced," open-mesh.org/projects/open-mesh/wiki

Scan for full code & documentation